Class 3 Notes

CAP Theorem, PACELC & the Consistency Spectrum — how distributed databases really trade off correctness, availability, and speed

1. First, the Basics: Vertical vs Horizontal Scaling

When one machine can no longer handle your load, you have two options:

- **Vertical scaling (scale up):** buy a bigger machine — more CPU, RAM, disk. Simple: your code and your database don't change. But it has a hard ceiling (the biggest machine money can buy), cost grows non-linearly, and it remains a single point of failure — when that one machine dies, everything dies.
- **Horizontal scaling (scale out):** add more machines and spread the work across them. There's no ceiling, capacity grows roughly linearly with cost, and losing one machine doesn't take you down. This is how every large system — Google, Amazon, Flipkart — is built.

But horizontal scaling has a price: the moment your data and computation live on multiple machines connected by a network, you have a distributed system — and you inherit every failure mode the network has.

2. System Design Is Mostly the Study of Distributed Trade-offs

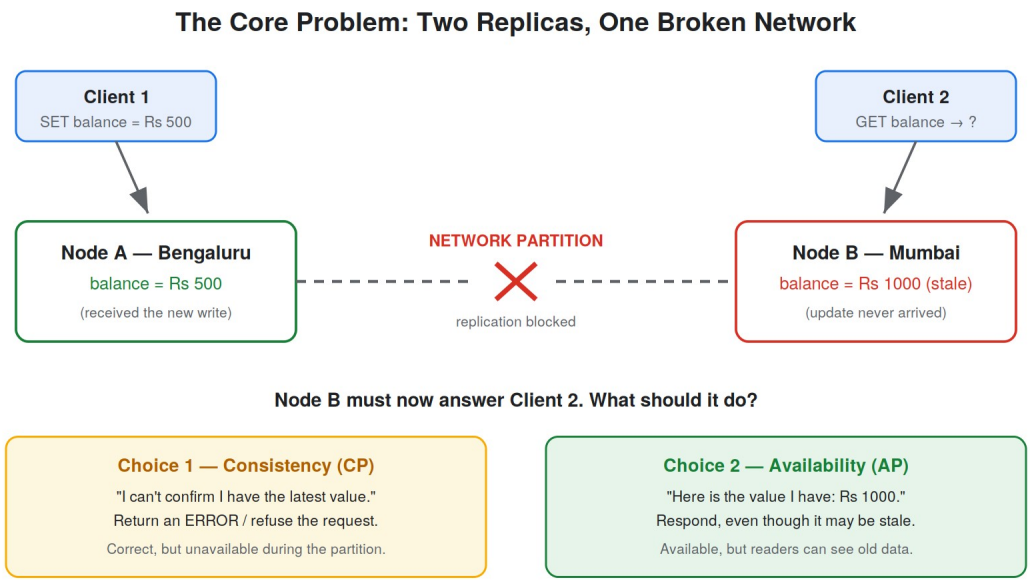
Almost every topic in system design — replication, sharding, caching, load balancing, queues, consensus — exists because we scaled horizontally. And in a distributed system, two things are always true:

1. Machines and networks fail, independently and unpredictably.
2. Information takes time to travel between machines — there is no instantaneous “shared truth.”

Consequently there are very few perfect solutions; there are only trade-offs. CAP and PACELC are the two most famous frameworks for reasoning about the central trade-off: correctness of data vs availability and speed of the system. We replicate data across nodes for three reasons — to survive machine failures, to serve reads at scale, and to keep data close to users. Replication is exactly what creates the problem below.

3. The Core Problem

Two replicas hold a user's bank balance. A client updates the balance on Node A. Before A can replicate the change to B, the network between them breaks — a partition. Now another client reads from Node B:



There is no third option. B cannot ask A for the latest value — the network is down. Every distributed database in the world must pick a side of this diagram (or let you pick, per operation). That forced choice is the CAP theorem.

4. The CAP Theorem

Proposed by Eric Brewer in a keynote in 2000 and formally proven by Gilbert and Lynch in 2002. It concerns three properties, each of which the proof defines precisely — and the precise definitions matter more than the acronym:

- **C — Consistency:** every successful read sees the latest committed write (or gets an error). Formally this is linearizability: even though the data lives on many nodes, the system behaves as if there were a single copy, updated at a single instant in time.
- **A — Availability:** every request received by any non-failing node gets a (non-error) response. Note what it does NOT promise: the response need not contain the latest data, and there is no time bound on it.
- **P — Partition tolerance:** the system keeps operating even when the network splits nodes into groups that cannot talk to each other.

> **The correct statement of CAP**
>
> "When a network partition occurs, a distributed system must choose between Consistency and Availability."
>
> It is NOT "pick any 2 of 3." P is not a feature you shop for — it is the failure scenario. The only real decision is what your system does during that failure: reject requests (CP) or serve possibly-stale data (AP).

5. What “P” Really Means (and Why You Can't Pick “CA”)

This is the most misunderstood letter. A partition is any situation where live nodes cannot reach each other — the definition says nothing about a physically cut cable. In practice, all of the following look exactly like a partition to your database:

- Network switch/router failure, or a power distribution unit taking out a rack's networking
- Network congestion severe enough that messages time out
- Long garbage-collection pauses — a JVM node frozen for 30 seconds is unreachable, which combined with timeouts is indistinguishable from a partition
- Firewall or network misconfigurations
- Plain software bugs in the networking stack

None of these are exotic. They happen constantly, even inside a single data center on a LAN. So “sacrificing P” would mean asserting “partitions won't happen to me” — which is only true for a single-node system, and a single-node system isn't a distributed database at all. That's why there is no meaningful “CA” distributed system: if you run more than one node over a network, P is forced on you, and the only question left is C-or-A during the partition.

> **Doubt to kill early: “Isn't a crashed node a partition?”**
>
> No. A dead node is a node failure; a partition is live nodes that can't reach each other (e.g. nodes {1,2} can talk to each other but not to {3,4,5}, and vice versa). They can look identical from the outside — which is precisely why systems must be designed for partitions: you often can't tell the difference in the moment.

6. CAP Availability ≠ High Availability (A and P Are Different Things)

Two separate confusions hide here. First: P is a condition (the network broke — something that happens to you), while A is a promise (how your system behaves — something you choose). They are not two flavors of the same thing.

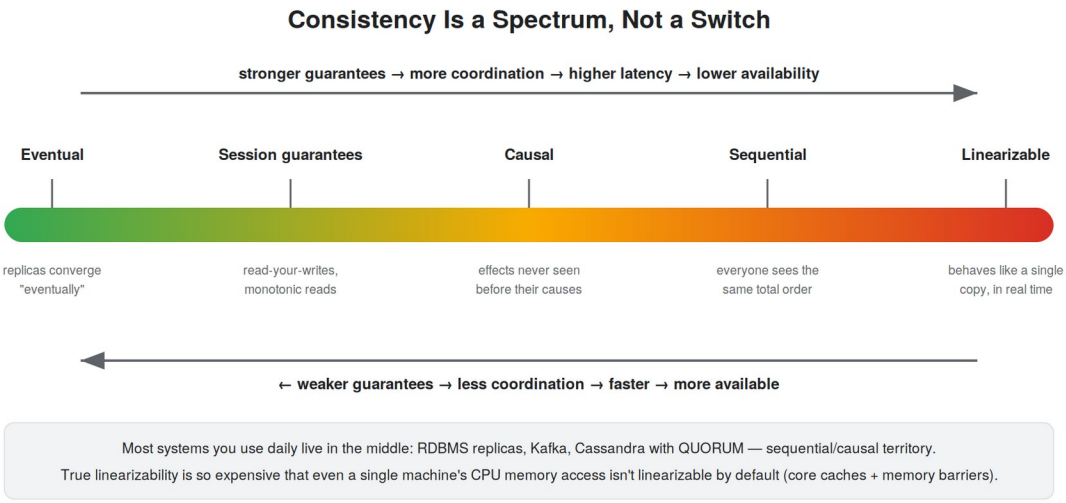
Second, and more subtle: CAP's “Availability” is not the “high availability” we care about in production:

	CAP Availability	High Availability (what business wants)
Scope	Every individual non-failing node must respond	The system as a whole responds
Latency	No time bound at all — a response after 10 minutes still counts	Useless without a latency bound; too slow = down
Correctness	Response may be arbitrarily stale	Response should be useful
Measured as	A property in a proof	Uptime: 99.9%, 99.99%, error budgets, SLOs

Two examples that make the gap obvious. A key-value store that throws every write away and answers every read with “empty” is perfectly CAP-available — and perfectly useless. Conversely, a database that fails over to a standby in 10 seconds when the primary dies is highly available in the practical sense, yet violates CAP availability (during those 10 seconds, some non-failing nodes refused requests). So “the system is operating” and “the system is CAP-available” are different claims — what you actually need in production is high availability with a latency bound, which CAP doesn't even talk about. This is the first big crack in CAP as a practical tool.

7. Consistency Is a Spectrum — and True Consistency Is Nearly Unattainable

CAP's “C” is linearizability, one of the strongest consistency models there is. But between “anything goes” and “perfect single-copy behavior” lies a whole ladder of models:



- **Linearizable:** the system behaves as if there is exactly one copy of the data, and every operation takes effect at one instant in real time. Once any client sees a value, every later read (from anyone, anywhere) sees it too.
- **Sequential:** all clients observe writes in the same order, but that order need not match real time. Most “strongly consistent” setups (two-phase commit, synchronous replication) actually land here, not at linearizable.
- **Causal:** if write B was caused by (or done after seeing) write A, nobody ever observes B without A. Unrelated writes may be seen in different orders.
- **Session guarantees:** read-your-own-writes (you always see your own updates) and monotonic reads (you never see time go backwards). Cheap, and often all a product actually needs.
- **Eventual:** if writes stop, all replicas converge to the same value — no promise about when, or what you read in between.

How hard is true linearizability? Consider that even a single machine doesn't give it to you by default. Each CPU core has its own caches; cores communicate through shared memory like a tiny distributed

system, and memory access across cores is NOT linearizable unless you explicitly use memory-barrier instructions — which is exactly what Java's 'volatile' keyword inserts. If one computer needs special instructions to act like “one copy of the data,” imagine the coordination cost across machines in different cities. That's why almost no real database offers linearizability by default — and why we've happily built decades of good software on weaker levels.

> **Side note — don't confuse this with SQL isolation levels**
>
> MySQL's read-committed / repeatable-read / serializable are ISOLATION levels: they govern how concurrent transactions interleave on one system. The consistency spectrum above governs how replicas agree with each other. Related, but different axes — 'serializable' is not 'linearizable'.

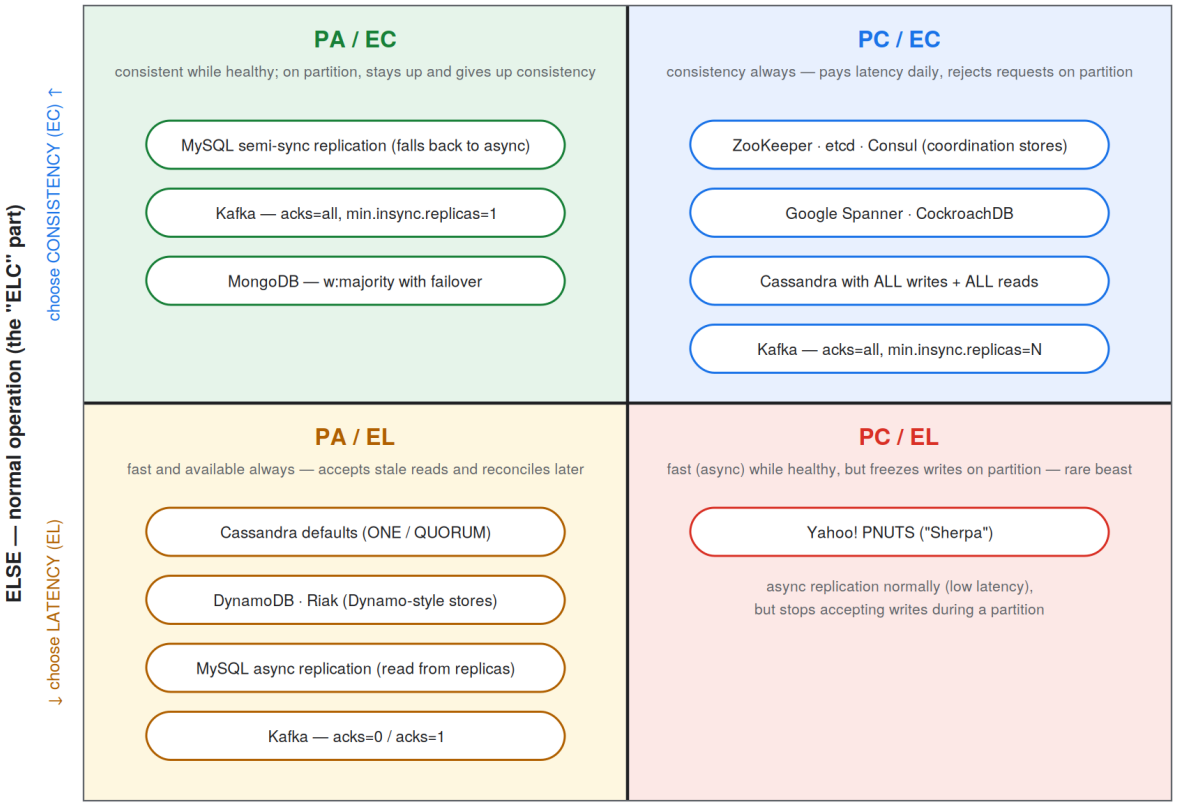
8. PACELC — the Better Lens

CAP has a second, bigger blind spot: it only describes the failure moment. Partitions are (hopefully) rare — what trade-off is your database making the other 99.9% of the time, when the network is healthy? Answer: latency vs consistency. Consider a write with two replicas and no partition:

- **Synchronous replication (choose consistency):** Node A waits until Node B has the write before acknowledging. Reads from either node are consistent — but every write pays a network round trip: ~2 ms becomes ~20 ms across a region.
- **Asynchronous replication (choose latency):** Node A acknowledges immediately and replicates in the background. Writes are fast — but for a moment A says Rs 500 while B says Rs 1000.

PACELC (pronounced “pass-elk”) captures both regimes in one sentence: if there is a Partition, trade off Availability vs Consistency; Else, trade off Latency vs Consistency. Notice consistency appears on both axes — you pay for it during failures with availability, and during normal operation with latency. In fact the two axes are secretly one: unavailability is just latency with no upper bound.

The PACELC Quadrants — Where Real Databases Sit



Note how the same database appears in multiple quadrants — consistency is a configuration knob, not a fixed property.

9. Reading the Quadrants: Real Databases, Real Configurations

The most important lesson from the diagram: a database is not permanently “CP” or “AP.” Most modern stores expose knobs, and different configurations of the same product land in different quadrants.

System & configuration	PACELC	Why
MySQL, async replication (common setup)	PA / EL	Writes ack from the master immediately (latency over consistency); during a partition you can still write to the master and read stale data from replicas.
MySQL, semi-sync replication	PA / EC	Normally waits for one replica synchronously (consistency); if it can't reach any replica, it falls back to async and keeps serving (availability).
ZooKeeper / etcd / Consul	PC / EC	Built for locks, leader election, configuration — correctness is the entire product. The minority side of a partition refuses requests.
Cassandra, writes=ANY + hinted handoff	extreme PA / EL	Any node accepts any write and forwards it later. Maximum availability — but hints expire, so acknowledged writes can be lost. The price of extreme AP.
Cassandra, QUORUM reads + QUORUM writes	PA / EL (leaning C)	R + W > N: a read overlaps the latest write, so you usually read fresh data. Still not linearizable (last-write-wins timestamps), and a minority partition side is unavailable.
Cassandra, reads=ALL + writes=ALL	PC / EC	Every replica participates in everything. One dead node makes its keys unavailable — almost nobody runs this.
Kafka, acks=0 or acks=1	PA / EL	Producer doesn't wait (or waits only for the leader). Fast, available, can lose data on failover.
Kafka, acks=all + min.insync.replicas=N	PC / EC	All in-sync replicas must confirm; if too few are alive, writes are rejected. Consistency over availability.
DynamoDB / Riak (Dynamo-style)	PA / EL	Designed for shopping carts and catalogs: always accept the write, reconcile conflicts later (read repair, vector clocks).
Google Spanner / CockroachDB	PC / EC	Globally consistent transactions; pays coordination latency on every commit and rejects requests on the wrong side of a partition.
Yahoo! PNUTS	PC / EL	The odd quadrant: async replication normally (latency), but freezes writes during a partition (consistency). Proof all four quadrants exist.

> **Quorum intuition (worth memorizing)**
>
> N replicas, W nodes must ack a write, R nodes must answer a read. If R + W > N, every read set overlaps every write set, so a read touches at least one node with the latest write — strong-ish consistency

without waiting for everyone.

Example: N=3, W=2, R=2. But note: a partition side holding only 1 of 3 nodes has no quorum — it becomes unavailable. Quorum buys consistency by giving up availability on minority sides, and it still isn't full linearizability.

10. Choosing C vs A Is a Business Decision, Not an Engineering One

Engineers cannot vote on this in a design review. The right questions go to the business:

- "Would you rather be DOWN, or show WRONG data?" (partition: A vs C)
- "Would you rather be SLOWER on every request, or occasionally show stale data?" (normal operation: C vs L)

Most businesses, when asked honestly, choose availability — downtime is total loss of business, while inconsistency can usually be worked around after the fact:

- **Eventual consistency + read repair:** Dynamo-style systems reconcile divergent replicas when they're read.
- **Compensating transactions:** ATMs allow withdrawals (up to a limit) even when they can't reach the bank's network to verify your balance — and the bank charges overdraft fees later if you overdrew. Accept the inconsistency, fix it with a follow-up transaction.
- **Append-only records + reconciliation:** "accountants don't use erasers" — financial systems record everything and reconcile/audit later rather than blocking on perfect agreement.

But not always. Flipkart's flash sales — 10,000 units, 500,000 buyers arriving within seconds — chose consistency over availability: overselling 50,000 orders against 10,000 units of stock is far more expensive than being briefly unavailable and running the sale an hour later. Inventory reservation, payments, distributed locks, leader election: whenever double-processing is catastrophic, consistency wins. Compare with our Class 2 weather platform: a temperature reading being minutes stale hurts nobody, so we happily chose eventual consistency for availability, latency, and simpler pipelines.

11. Common Doubts, Answered

Doubt	Answer
"Can a distributed database be CA?"	No. P isn't optional — networks fail. "CA" is only meaningful for a single-node system, which isn't distributed.
"If partitions are rare, is CAP irrelevant most of the time?"	Yes — and that's exactly why PACELC exists. The everyday trade-off is latency vs consistency; you pay for consistency on every single request, partition or not.
"Does choosing AP mean I lose data?"	Usually not — AP means readers may see stale data while replicas converge. But extreme AP configs (Cassandra ANY + expired hints, Kafka acks=0) genuinely can lose acknowledged writes. Staleness and data loss are different failures; know which one your config risks.
"Is eventual consistency just 'no	No. It guarantees convergence (replicas agree once writes stop) but not

Doubt	Answer
guarantee'?"	freshness. Conflicts need a resolution rule: last-write-wins, vector clocks, or merging (the cart with shoes on your phone and a shirt on your laptop becomes [shoes, shirt]).
"Do quorum reads/writes give me full consistency?"	They give you read-latest-write in the common case (R+W>N), which is strong but not linearizable — and the minority side of a partition becomes unavailable.
"My database supports 'serializable' — isn't that the strongest?"	Serializable is an isolation level (transactions on one system); linearizable is a consistency model (agreement across replicas). Different axes. Neither implies the other.
"A node crashed — is that a partition?"	A crash is a node failure; a partition is live nodes unable to reach each other. They look identical from outside (which is why you must design for partitions), but CAP's availability clause specifically talks about non-failing nodes.
"The system is up and responding — so it's Available, right?"	"Up" is high availability (whole-system, time-bounded, what SLOs measure). CAP availability is a different, stricter-and-weirder property: every non-failing node responds, with no latency bound. A cluster mid-failover is practically available but violates CAP-A.
"In an interview: is Cassandra CP or AP?"	The winning answer: "It's tunable — per-query consistency levels move it between quadrants. With defaults it behaves PA/EL; with ALL/ALL it approaches PC/EC." Then state which config your design needs and why. Naming the trade-off beats naming the letter.
"Who decides all this?"	The business, informed by engineering: you present 'down vs wrong vs slow' with costs; they pick. Then you configure the database to match.

12. The One-Slide Summary

If you remember nothing else

1. Horizontal scaling ⇒ distributed system ⇒ network failures are yours now. Partition tolerance is not a choice.

2. CAP: WHEN a partition happens, choose Consistency (refuse requests) or Availability (serve possibly-stale data). It is not "pick 2 of 3."

3. CAP availability ≠ high availability, and CAP consistency (linearizability) is so strong that even one machine's CPU doesn't give it by default. Both C and A are spectrums; CP and AP are just the two endpoints.

4. PACELC: Partition → A vs C; Else → Latency vs C. You pay for consistency every day (latency), not just during failures.

5. Databases are configurable, not fixed: MySQL, Kafka, and Cassandra each live in multiple quadrants depending on settings.

6. The choice is a business decision: "down, wrong, or slow — pick your poison." Most pick availability and repair inconsistency later (read repair, compensating transactions); pick consistency when double-

> processing is catastrophic (inventory, payments, locks, leader election).